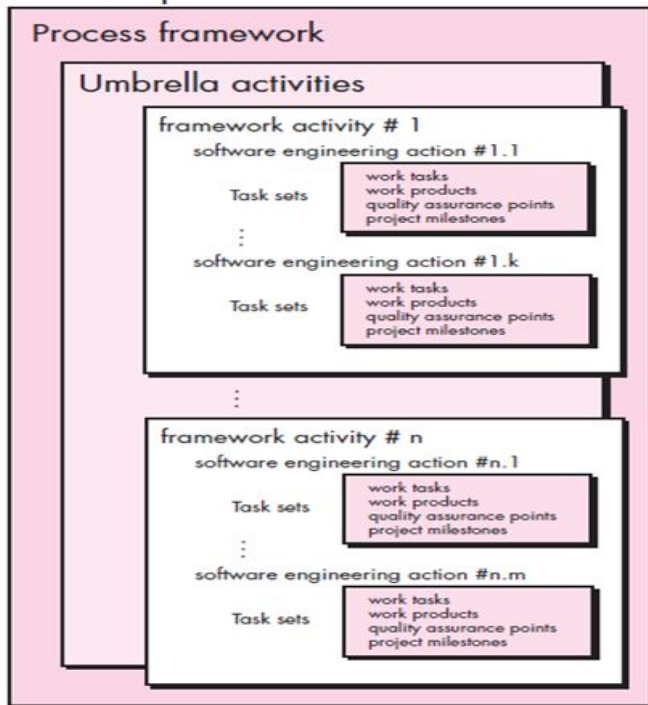


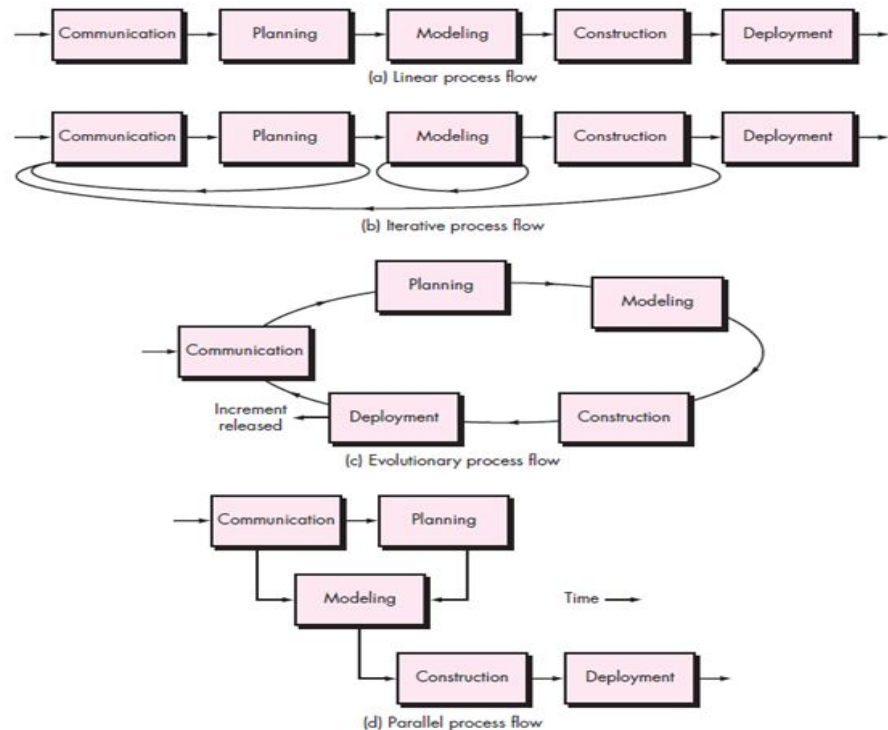
PROCESS MODELS

A GENERIC PROCESS MODEL

Software process



A software process framework



Process flow

SDLC Process Model

- ▶ SDLC – Software Development Life Cycle
- ▶ Process model is a framework that describes the activities, actions, tasks, milestones and work products performed at each stage of a software development project that leads to a high quality software

Life cycle models

- ▶ **Waterfall Model**
- ▶ **Incremental process Models**
 - Incremental model
 - RAD model
- ▶ **Evolutionary Process Models**
 - Prototyping model
 - Spiral model

PRESCRIPTIVE PROCESS MODELS

1. The Waterfall Model

FIGURE 2.3 The waterfall model

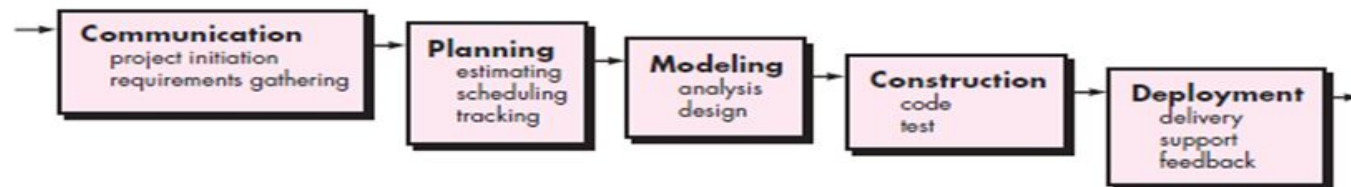
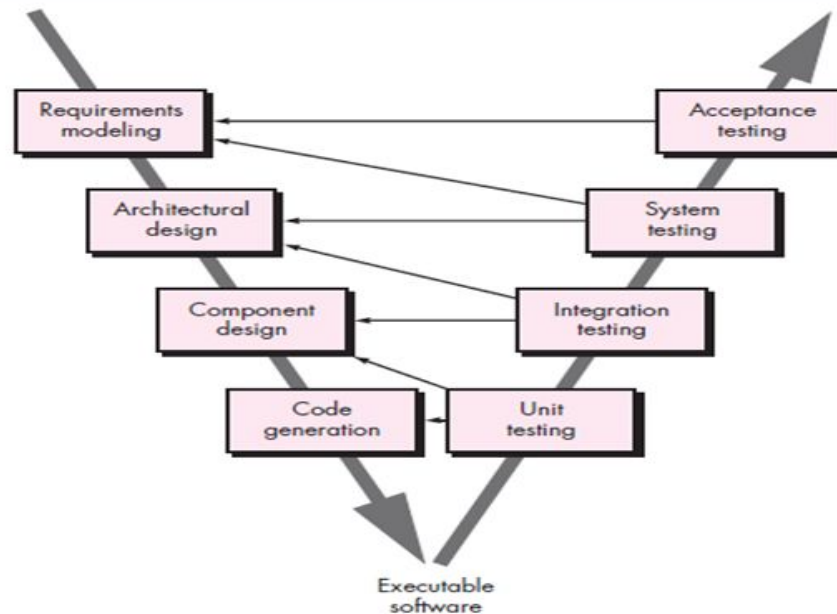
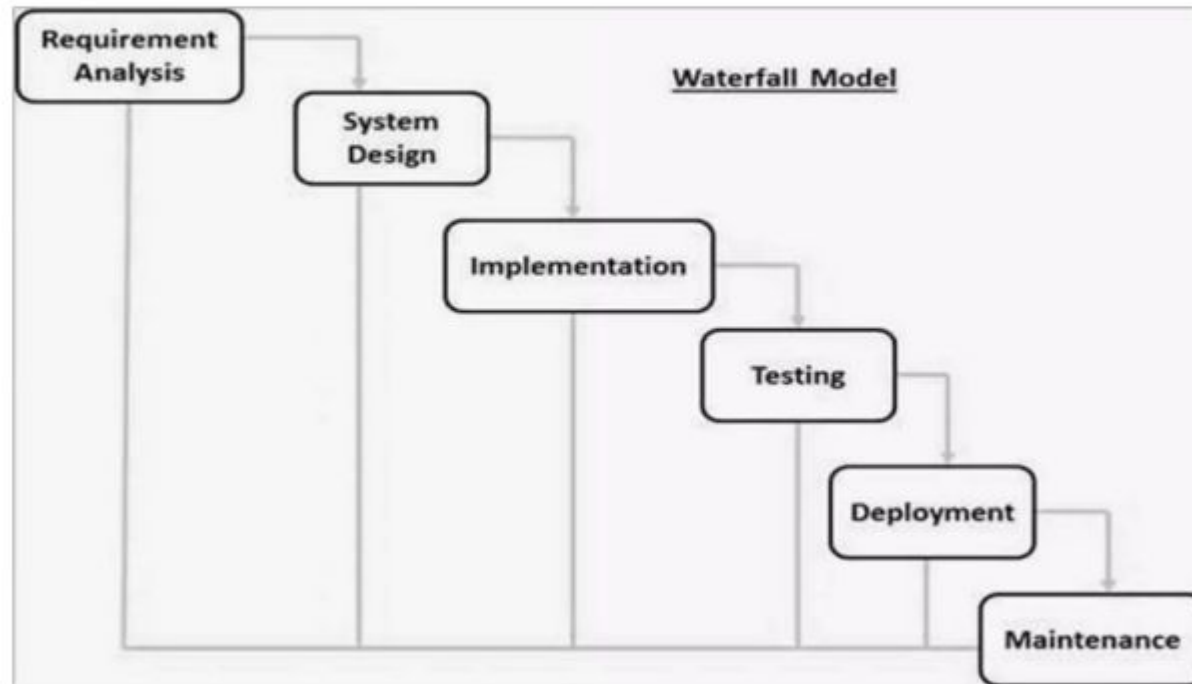


FIGURE 2.4
The V-model

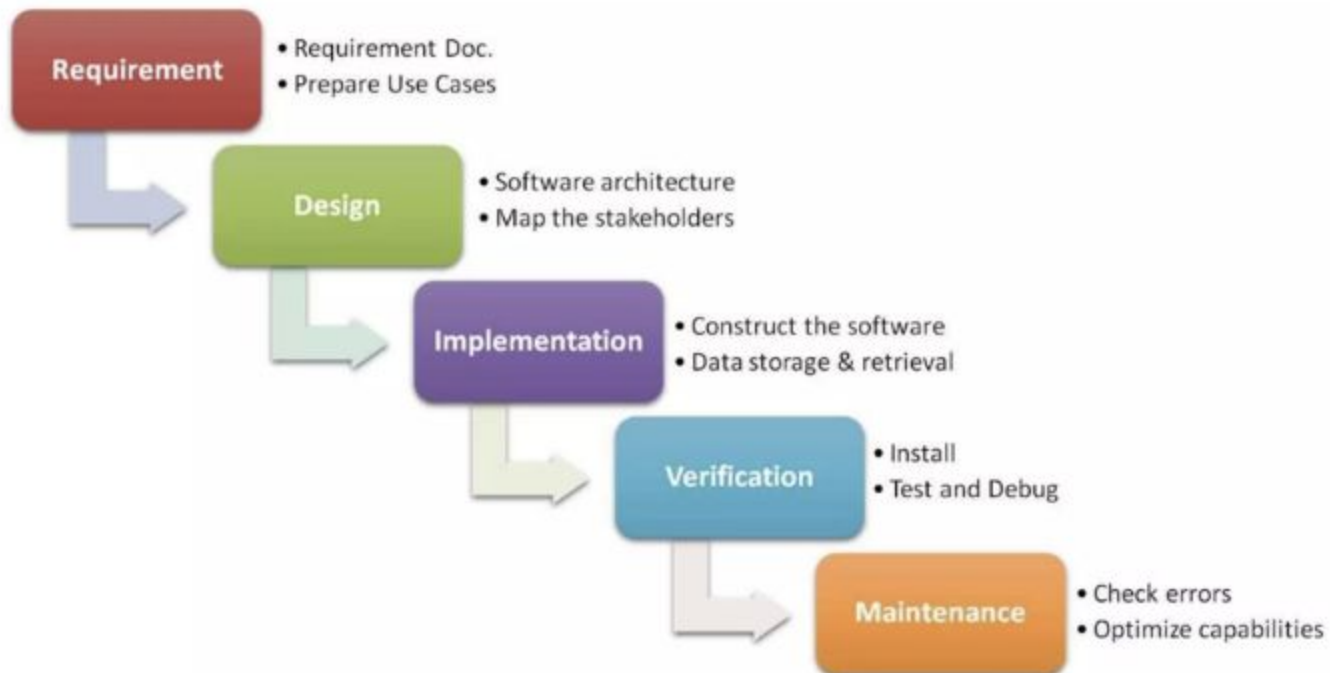


WATERFALL MODEL

Linear life cycle model or classic life cycle model



WATERFALL MODEL



WATERFALL MODEL

- ▶ **Project initiation & requirement gathering**
 - What is the Problem to Solve?
 - What Does Customer Need/Want?
 - Interactions Between SE and Customer
 - Identify and Document System Requirements
 - Generate User Manuals and Test Plans
- **Planning**
 - Prioritize the requirements
 - Plan the process

WATERFALL MODEL

- ▶ **Analysis and design**

- How is the Problem to be Solved?
- High-Level Design
- Determine Components/Modules
- Transition to Detailed Design
- Detail Functionality of Components/Modules

- ▶ **Coding and Testing**

- Writing Code to Meet Component/Module Design Specifications
 - Individual Test Modules in Isolation
 - Integration of Components/Modules into Subsystems
 - Integration of Subsystems into Final Program
- 

WATERFALL MODEL

▶ **Deployment**

- System Delivered to Customer/Market
 - Bug Fixes and Version Releases Over Time
- Strengths

- ▶ Easy to understand, easy to use
 - ▶ Provides structure to inexperienced staff
 - ▶ Milestones are well understood
 - ▶ Sets requirements stability
 - ▶ Good for management control (plan, staff, track)
 - ▶ Works well when quality is more important than cost or schedule
- 

Waterfall Drawbacks

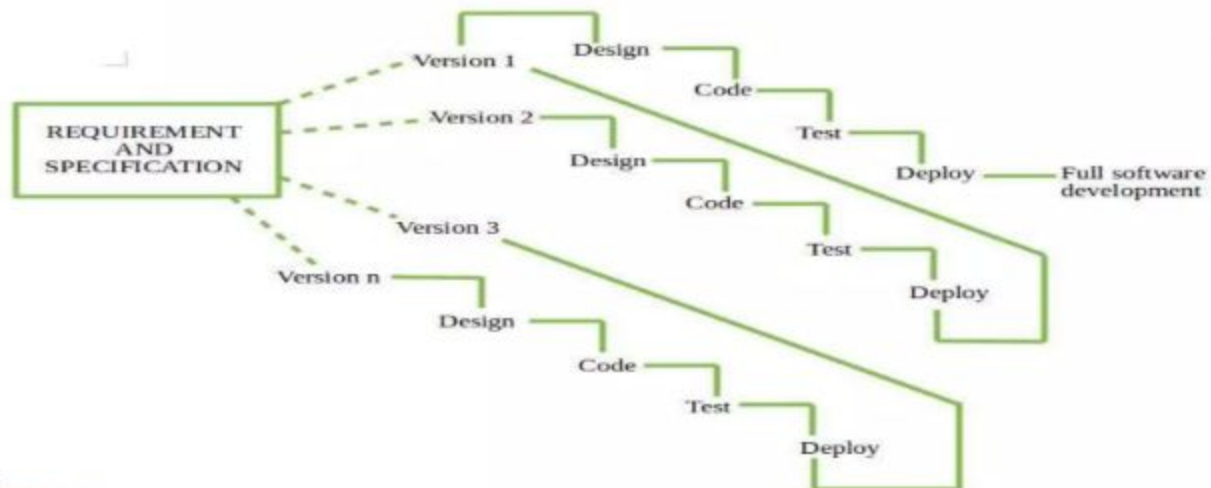
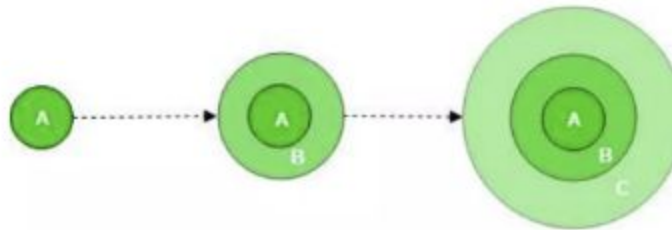
- ▶ All projects cannot follow linear process
- ▶ All requirements must be known upfront
- ▶ Few business systems have stable requirements
- ▶ The customer must have patience
- ▶ A working version of the program will not be available until late in the project time-span
- ▶ Leads to 'blocking states'
- ▶ Inappropriate to changes



Incremental SDLC Model

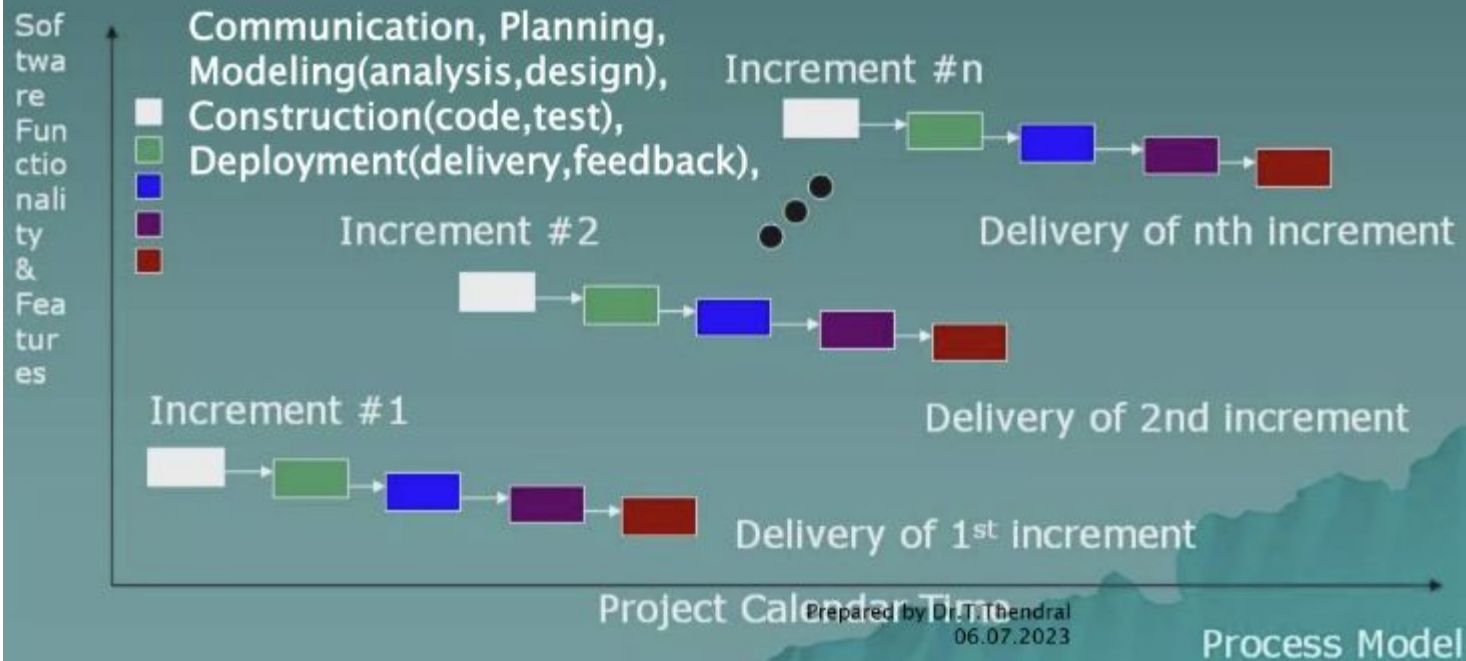
- ▶ Combines the elements of waterfall model in an iterative fashion
- ▶ Construct a partial implementation of a total system
- ▶ Then slowly add increased functionality
- ▶ User requirements are prioritised and the highest priority requirements are included in early increments
- ▶ Each subsequent release of the system adds function to the previous release, until all designed functionality has been implemented

INCREMENTAL MODEL



Incremental Process Model

- ◆ Combines the elements of the waterfall model applied in an iterative fashion.



- The **Incremental Approach** uses a set number of steps and development goes from start to finish in a linear path of progression.
- The **Iterative Approach** has no set number of steps, rather development is done in cycles.
- Incremental Model is a process of software development where requirements are broken down into multiple standalone modules of software development cycle.
- Incremental development is done in steps from analysis design, implementation, testing/verification, maintenance.



Incremental Model Weaknesses

- ▶ Requires good planning and design
- ▶ Requires early definition of a complete and fully functional system to allow for the definition of increments
- ▶ Well-defined **module interfaces** are required (some will be developed long before others)
- ▶ Total cost of the complete system is not lower

When to use the Incremental Model

- ▶ When staffing is not available by deadline
- ▶ Most of the requirements are known up-front but are expected to evolve over time
- ▶ When the software can be broken into increments and each increment represent a solution
- ▶ A need to get basic functionality to the market early
- ▶ On projects which have lengthy development schedules
- ▶ On a project with new technology

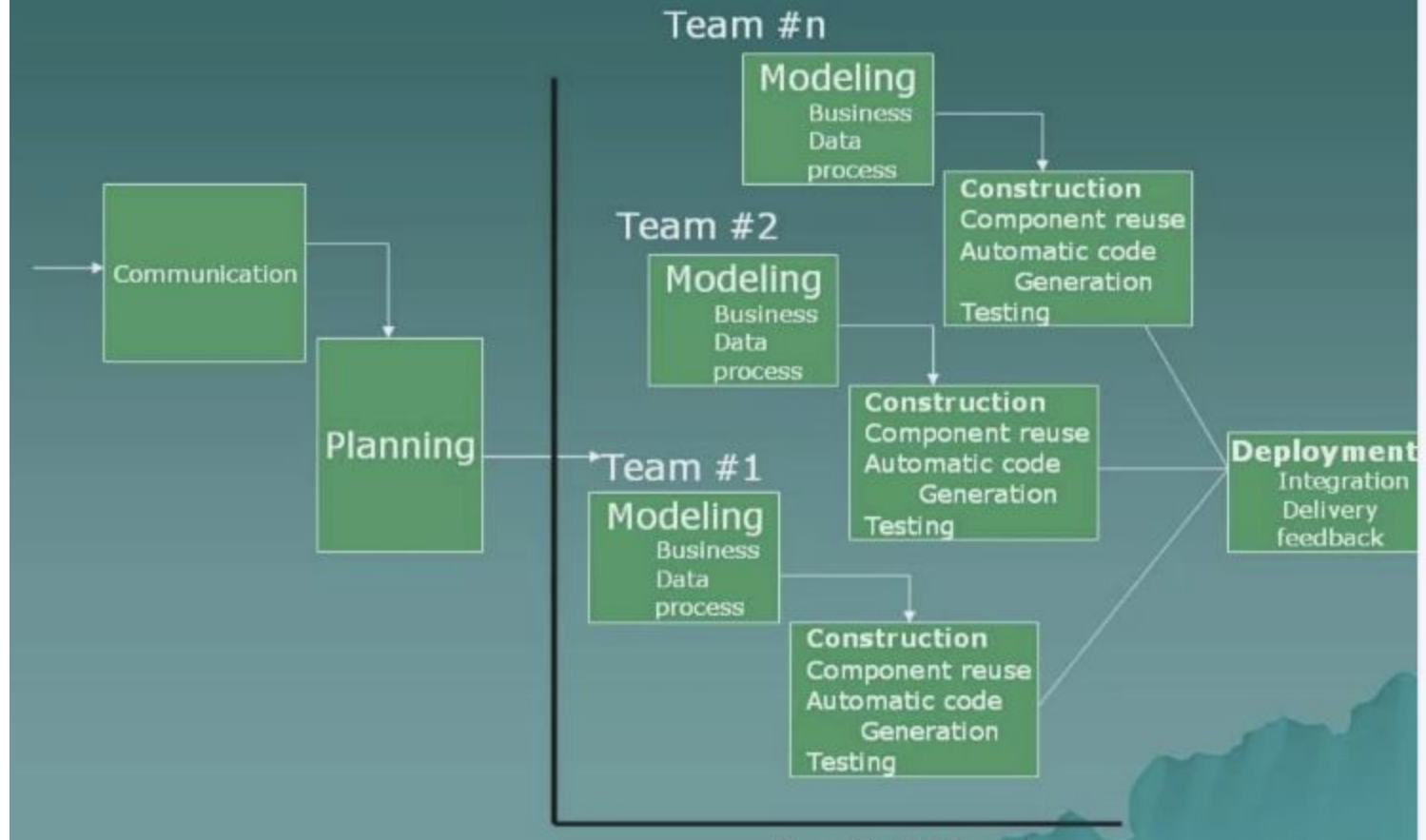


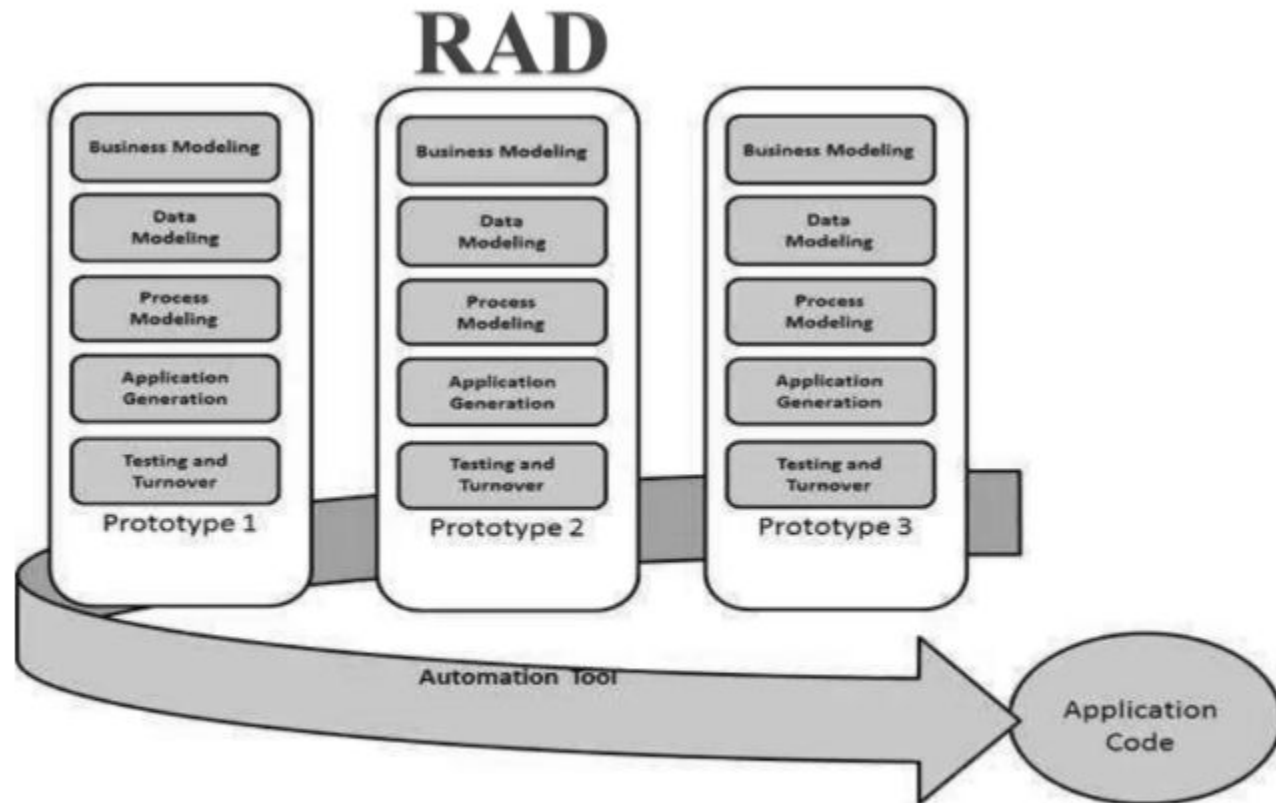
RAD MODEL

Rapid Application Development Model

- ▶ An incremental process model that emphasizes short development cycle
- ▶ “High-speed” adaptation of the waterfall model.
- ▶ RAD approach also maps into the generic framework activities

RAD Model





- ▶ Business Modeling-flow of information and the distribution of information between various business channels.

Advantages of RAD

- Changing requirements can be accommodated
- Progress can be measured
- Iteration time can be short with use of powerful RAD tools
- Productivity with fewer people in a short time
- Reduced development time
- Increases reusability of components
- Quick initial reviews occur
- Encourages customer feedback
- Integration from very beginning solves a lot of integration issues



Disadvantages in RAD

- Requires highly skilled developers/designers.
- High dependency on modeling skills.
- Inapplicable to cheaper projects as cost of modeling and automated code generation is very high.
- Management complexity is more.
- Suitable for systems that are component based and scalable.
- Requires user involvement throughout the life cycle.
- Suitable for project requiring shorter development times.



When to use RAD

- ▶ Reasonably well-known requirements
- ▶ User involved throughout the life cycle
- ▶ Project can be time-boxed
- ▶ Functionality delivered in increments
- ▶ High performance not required
- ▶ Low technical risks
- ▶ System can be modularized

▶ **Evolutionary Process Models**

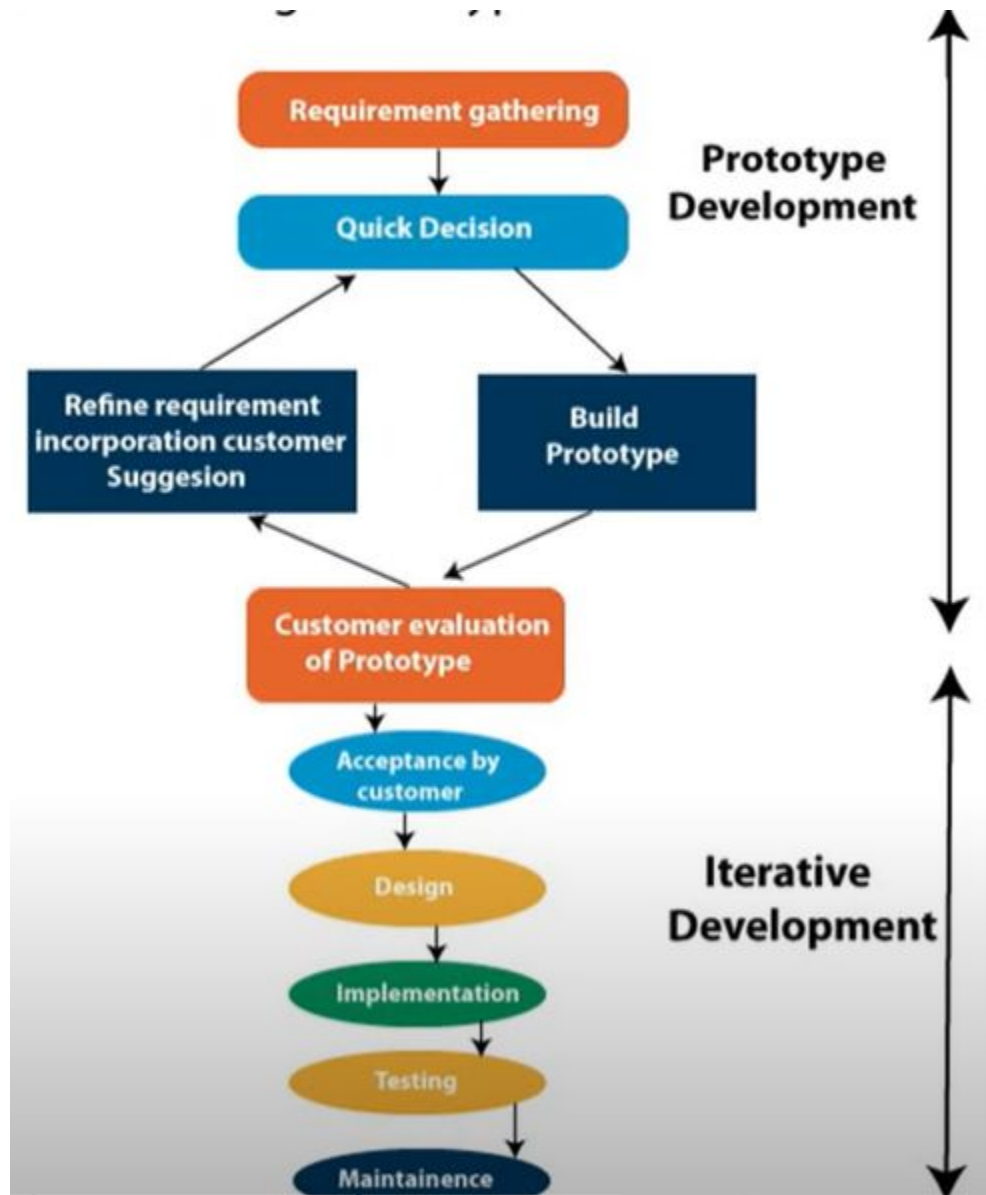
- Prototyping model
- Spiral model

EVOLUTIONARY PROCESS MODEL

- ▶ These models produce an increasingly more complete version of the software with each iteration
- ▶ When to use
 - Tight market deadlines
 - Well defined system requirements
 - No details about system definition

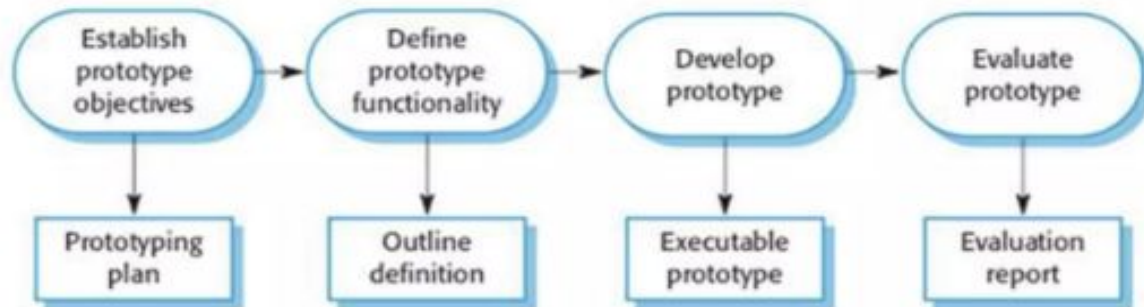
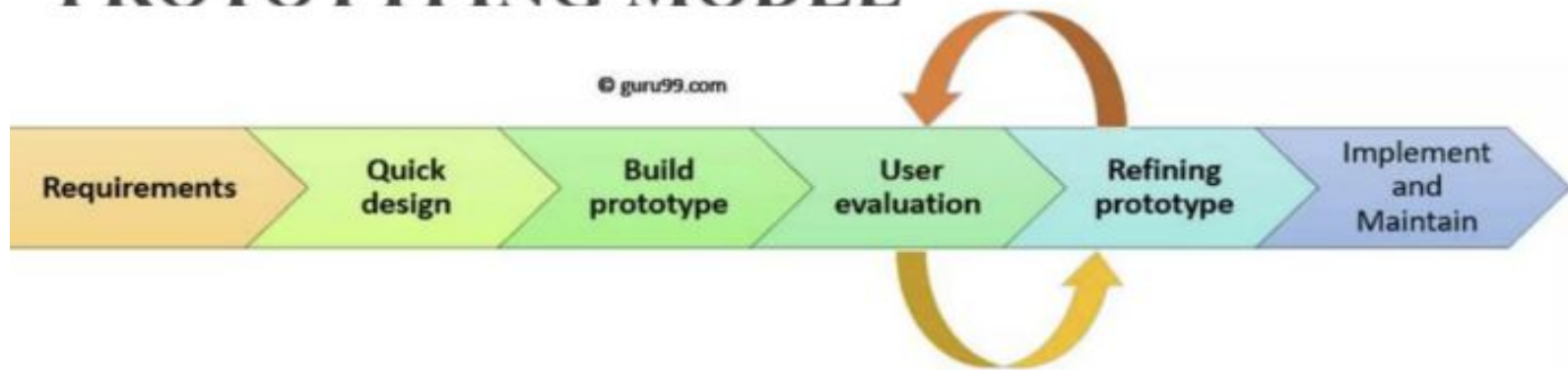
PROTOTYPING MODEL

- ▶ A prototype is a partially developed product that enables customers and developers to examine some aspects of the proposed system and decide if it is suitable or appropriate for the finished product
 - Start with what is known about requirements
 - Do a quick design
 - Build the prototype by focusing on what will be seen by the user
 - Use the prototype to show the user and help refining requirements
- 

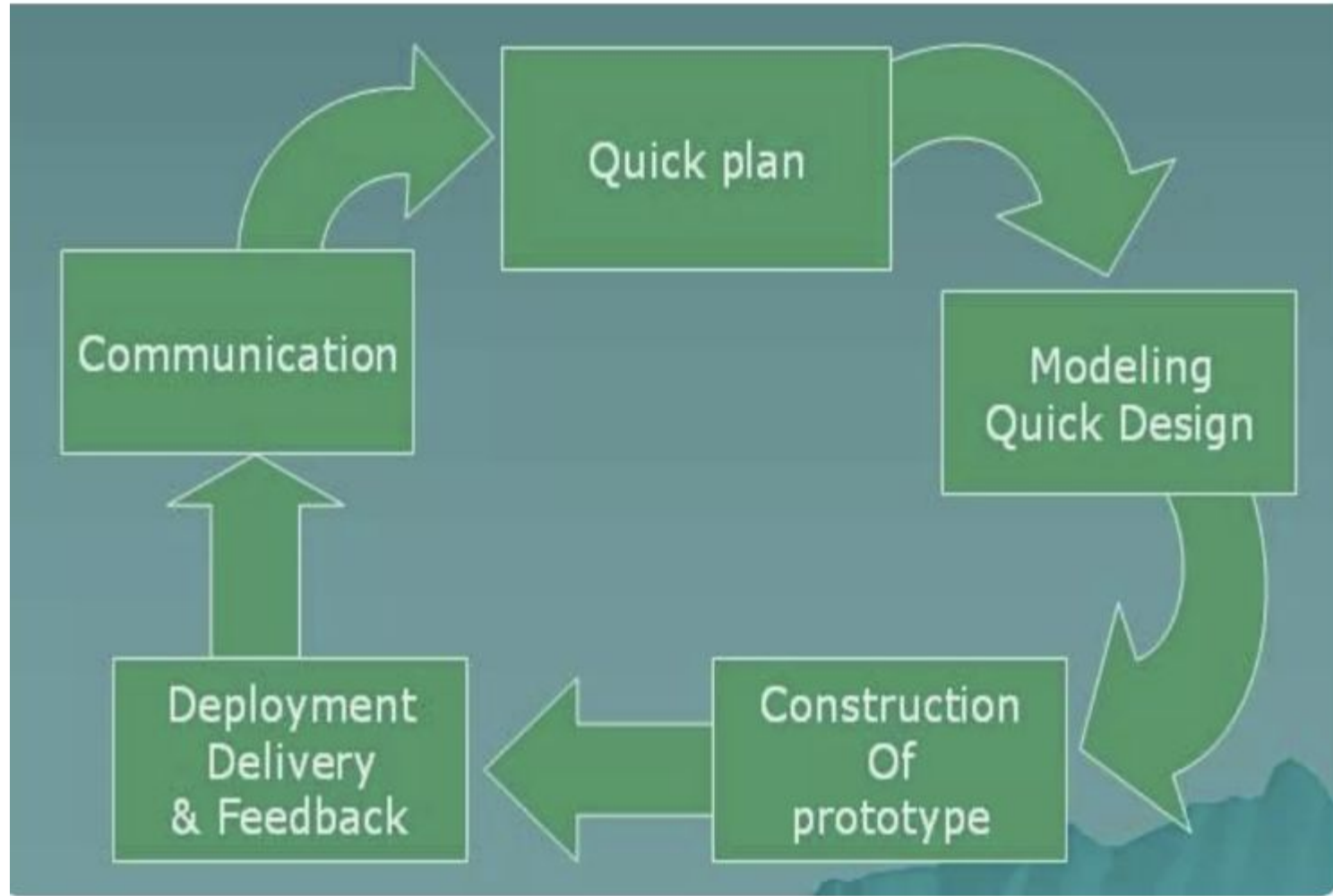


PROTOTYPING MODEL

© guru99.com



PROTOTYPING MODEL





Prototype

Actual



Step 1: Requirements gathering and analysis

- ▶ A prototyping model starts with requirement analysis
- ▶ During the process, the users of the system are interviewed to know what is their expectation from the system.

Step 2: Quick design

- ▶ The second phase is a preliminary design or a quick design
 - ▶ A simple design of the system is created
 - ▶ However, it is not a complete design
 - ▶ It gives a brief idea of the system to the user
 - ▶ The quick design helps in developing the prototype
- 

Step 3: Build a Prototype

- ▶ In this phase, an actual prototype is designed based on the information gathered from quick design

It is a small working model of the required system.

Step 4: Initial user evaluation

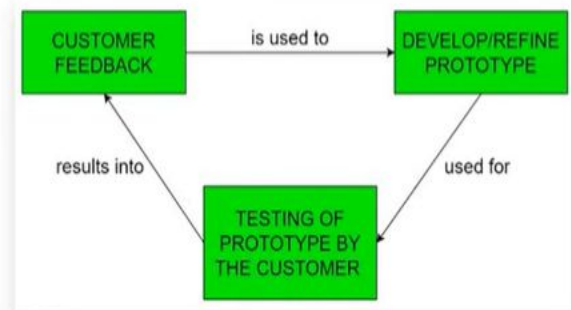
- ▶ In this stage, the proposed system is presented to the client for an initial evaluation.
- ▶ It helps to find out the strength and weakness of the working model.
- ▶ Comment and suggestion are collected from the customer and provided to the developer.

Step 5: Refining prototype

- ▶ If the user is not happy with the current prototype, you need to refine the prototype according to the user's feedback and suggestions.
- 

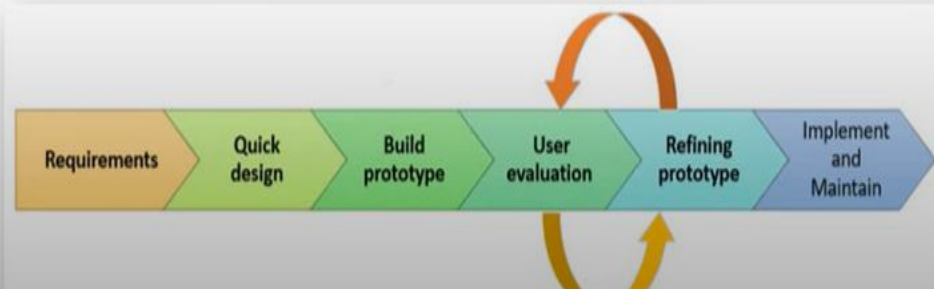
Advantages of Prototyping Model

1. Prototype model need not know the detailed input, output, processes, adaptability of operating system and full machine interaction.
2. Good where requirement are changing.
3. Customers are actively involved in development.
4. Prototypes can be changed and even discarded.
5. Errors can be detected much earlier as the system is made side by side.
6. Flexibility in design.
7. Helps team member to communicate effectively



Disadvantages of Prototyping Model

1. The client involvement is more and it is not always considered by the developer.
2. It is a slow process because it takes more time for development.
3. Many changes can disturb the rhythm of the development team.
4. Documentation is poor as the requirements change frequently.
5. It is a thrown away prototype when the users are confused with it.
6. Costly with respect to time & money.
7. Prototyping tools are expensive.



Drawbacks of Prototyping Model

- ▶ Customer sees what appears to be a working version of the software and presumes that it is the final thing
- ▶ The developer often makes implementation compromises in order to get a prototype working quickly

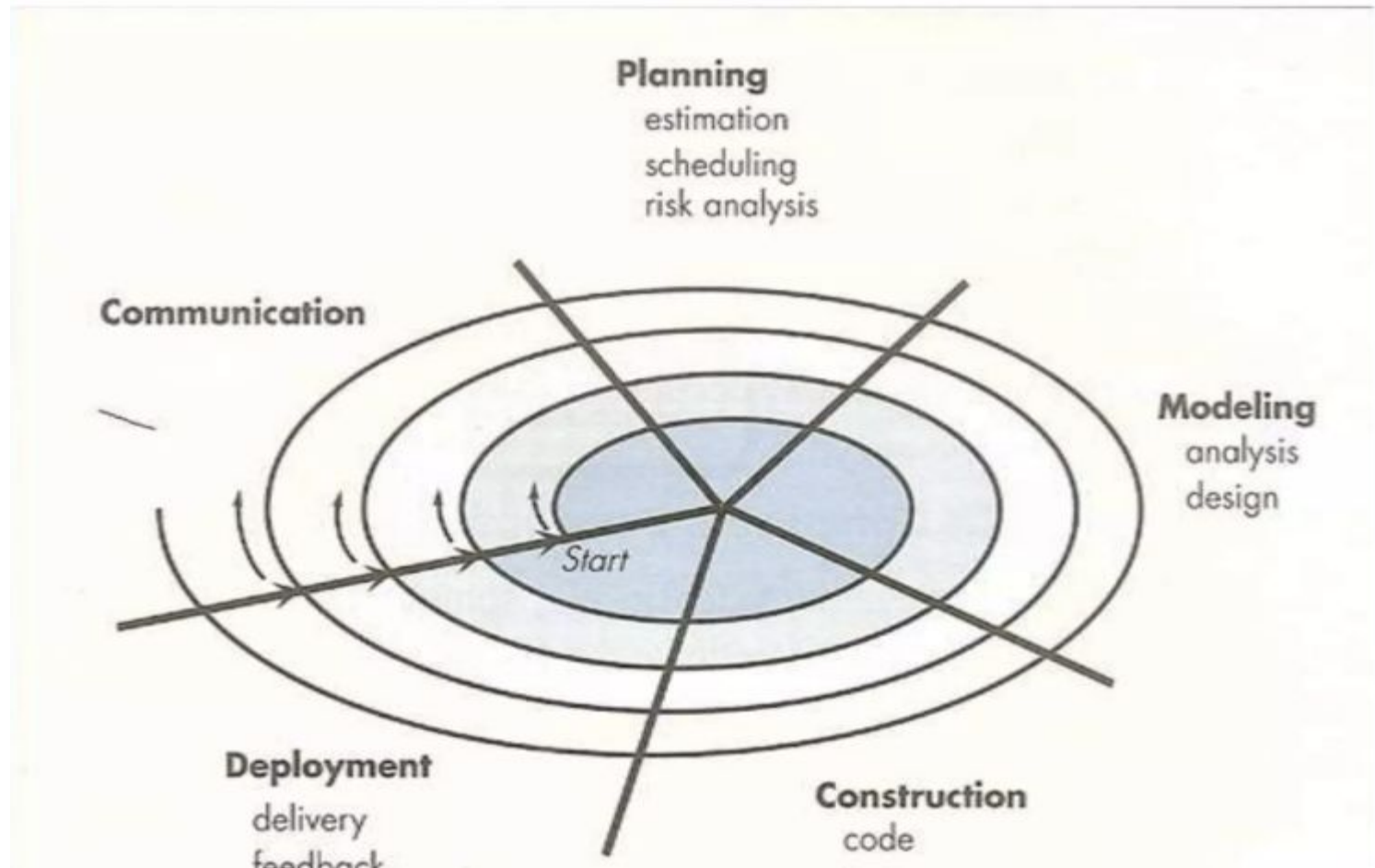
Only one advantage is actual software is engineered with an eye toward quality

SPIRAL MODEL

- ▶ It couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model Process
- ▶ Adapted to complete life cycle
- ▶ Process is represented as a spiral rather than as a sequence of activities with backtracking.
- ▶ Each loop in the spiral represents a phase in the process.
- ▶ No fixed phases such as specification or design - loops in the spiral are chosen depending on what is required.



SPIRAL MODEL



- First Spiral represents : Conceptual Development
- Second Spiral represents: System Development
- Third Spiral represents: System enhancement
- Fourth Spiral represents: System Maintains

Spiral Model Strengths

- ▶ Focuses attention on reuse options
- ▶ Focuses attention on early error elimination
- ▶ Puts quality objectives up front
- ▶ Integrates development and maintenance
- ▶ Provides a framework for hardware/software Development

Drawbacks of the Spiral Model

- ▶ It may be difficult to convince customers that the evolutionary approach is controllable
- ▶ It demands risk assessment expertise and relies on this expertise for success
- ▶ If a major risk is uncovered and managed, problems will occur

When to use Spiral Model

- ▶ When creation of a prototype is appropriate
- ▶ When costs and risk evaluation is important
- ▶ For medium to high-risk projects
- ▶ Long-term project commitment unwise because of potential changes to economic priorities
- ▶ Users are unsure of their needs
- ▶ Requirements are complex
- ▶ New product line
- ▶ Significant changes are expected

What is V-model

- ▶ V- model means Verification and Validation model.
- ▶ V-Shaped life cycle is a sequential path of execution of processes.
- ▶ Each phase must be completed before the next phase starts.
- ▶ Testing of the product is planned in parallel with a corresponding phase of development.



When to use the V-model?

- ▶ The V-shaped model should be used for small to medium sized projects where requirements are clearly defined and fixed.
- ▶ The V-Shaped model should be chosen when sample technical resources are available with needed technical expertise.

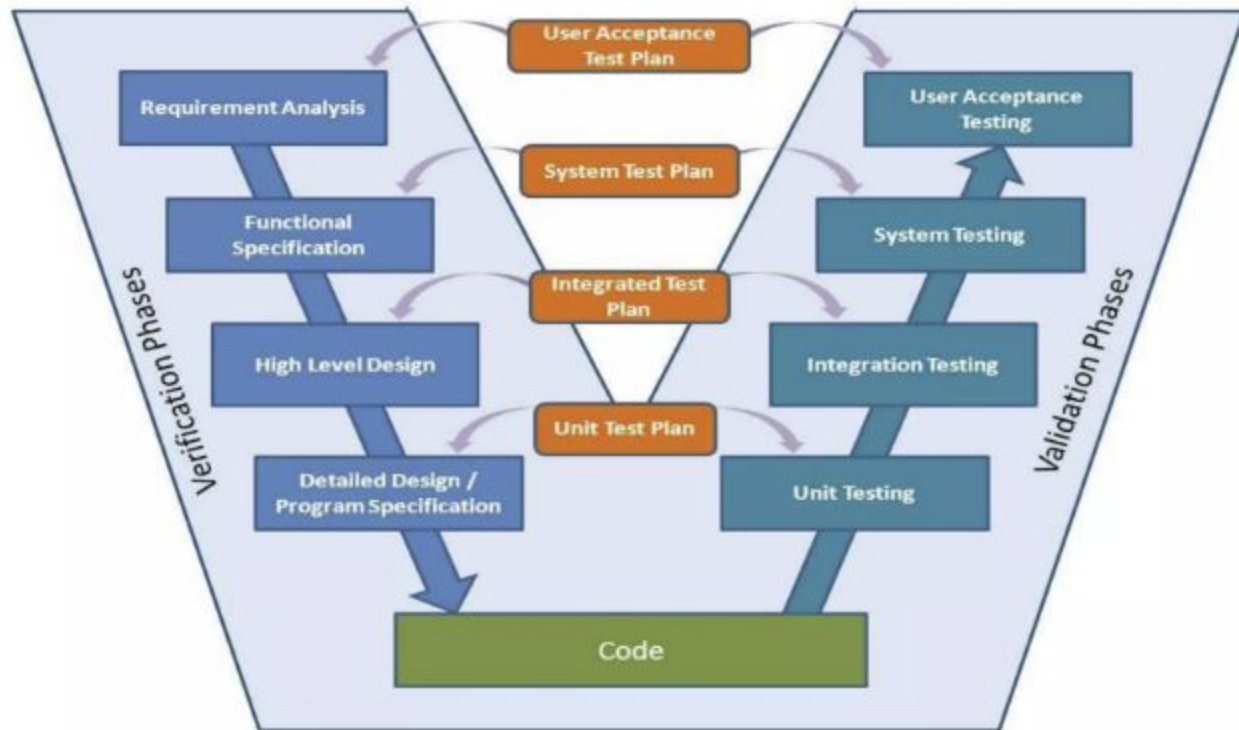


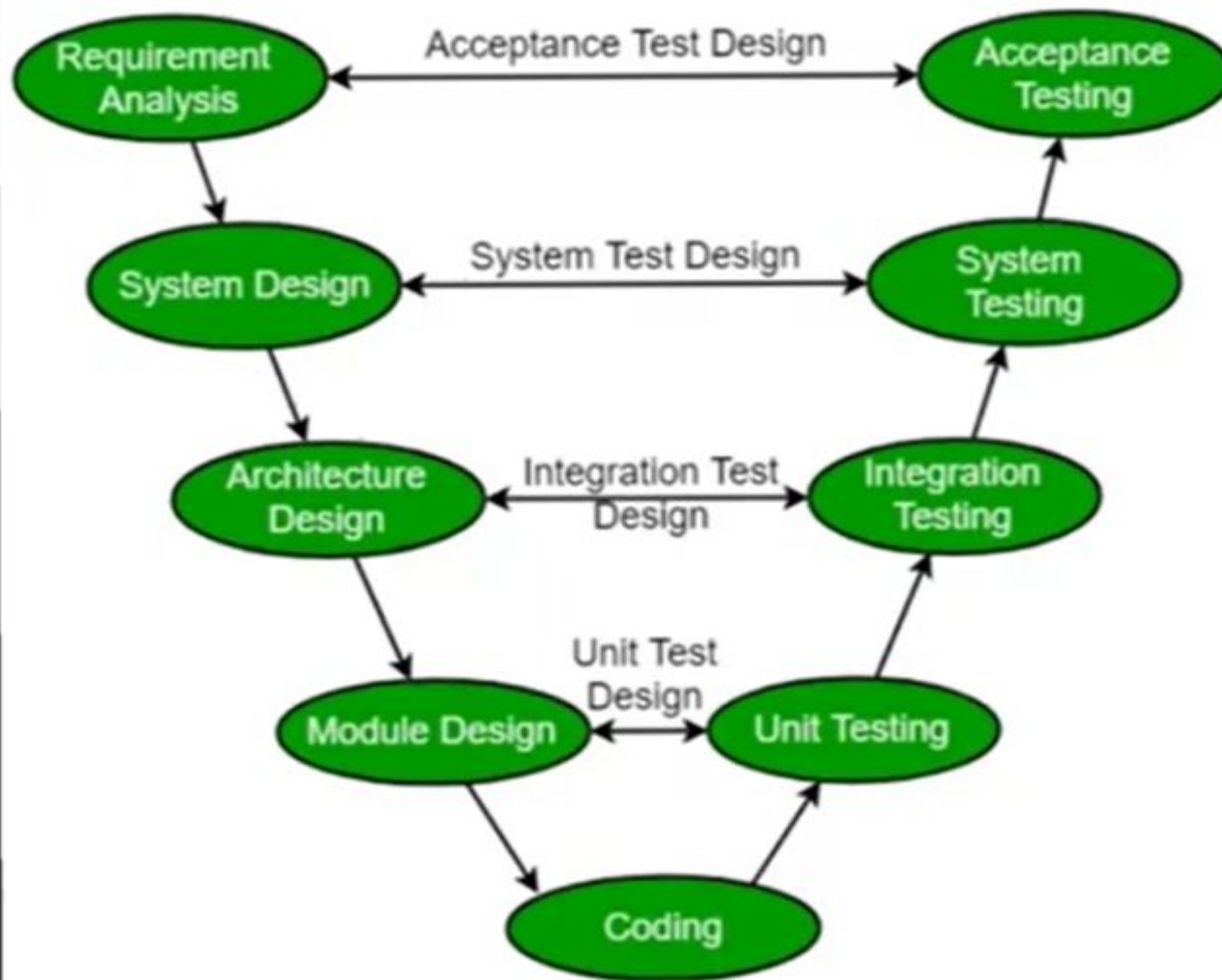
Phases

There are two phases

- Verification Phase
- Validation Phase

Pictorial view of v-Model





Verification Phase

- › **Requirements Analysis:**

the first step in the verification process, the requirements of the system are collected by analyzing the needs of the user.

- › **System design:**

In this phase system engineers analyze and understand the business of the proposed system by studying the user requirements document.



Validation Phase

- ▶ Unit testing

In the V-Model, Unit Test Plans (UTPs) are developed during module design phase. These UTPs are executed to eliminate bugs at code level or unit level. A unit is the smallest entity which can independently exist, e.g. a program module. Unit testing verifies that the smallest entity can function correctly when isolated from the rest of the codes/units.



Validation Phase

- **Integration testing**

Integration Test Plans are developed during the Architectural Design Phase. These tests verify that units created and tested independently can coexist and communicate among themselves.

- **System testing**

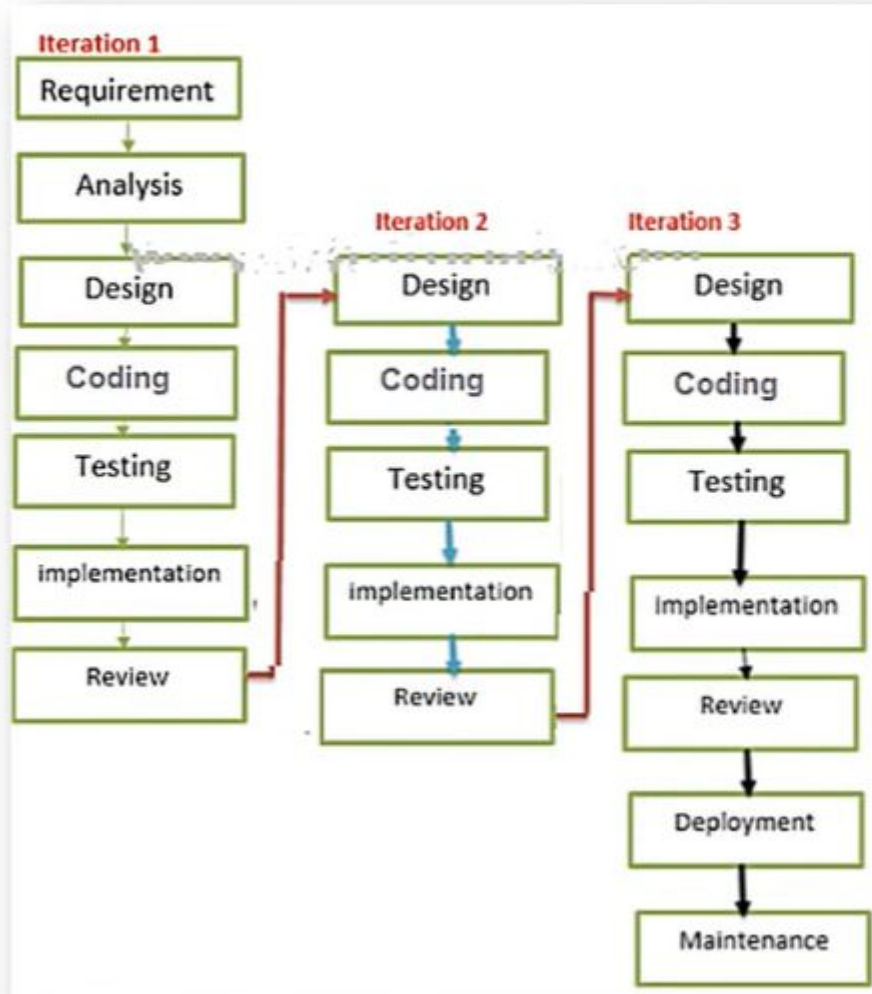
System Tests Plans are developed during System Design Phase. Unlike Unit and Integration Test Plans, System Test Plans are composed by client's business team. System Test ensures that expectations from application developed are met.

Validation Phase

- User acceptance testing

User Acceptance Test (UAT) Plans are developed during the Requirements Analysis phase. Test Plans are composed by business users. UAT is performed in a user environment that resembles the production environment, using realistic data.

Iterative Model



➤ Features:

- Changing Requirements.
- Used in Large Projects.
- Customer feedback taken.

➤ Disadvantages:

- More resources required.
- Deadline not confirmed.
- Over budget.